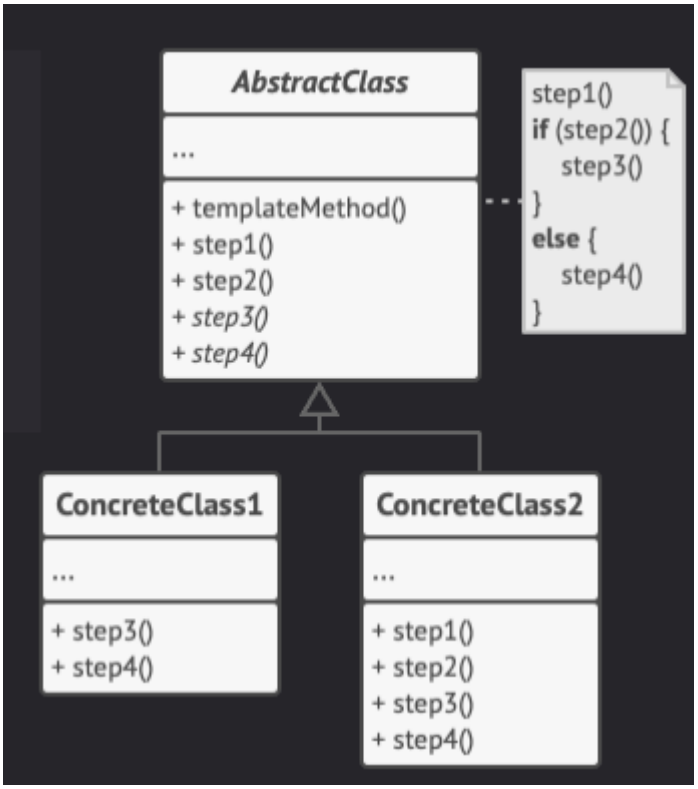
b) Investigación

- **Patrón Plantilla (Template Method):**
-

¿Qué especifica el patrón plantilla? Realizar el Diagrama de Clases. ¿Cómo se utilizaría el en las clases Controlador y clases de Servicio?

El patrón Método Plantilla (Template Method) es un patrón de diseño de comportamiento que define el esqueleto de un algoritmo dentro de un método de una clase base, delegando la implementación de algunos de sus pasos a las subclases. Esto permite que las subclases redefinan ciertas partes del algoritmo sin cambiar su estructura general ni el orden de ejecución.



Aplicación en un Modelo MVC

En una arquitectura Modelo-Vista-Controlador (MVC), el patrón Método Plantilla es extremadamente útil para estandarizar flujos de trabajo repetitivos en las capas de Controladores y Servicios, reduciendo la duplicación de código.

1) Controladores: Los controladores a menudo siguen un flujo estándar al recibir una petición web: validar permisos, extraer parámetros, ejecutar la lógica de negocio y devolver una vista o un JSON.Puedes crear un BaseController que defina el flujo y dejar que los controladores específicos implementen solo lo que cambia.

2) Servicios: En la capa de servicio, el patrón es útil al procesar importaciones de datos, generar reportes o manejar transacciones de pago, donde el ciclo de vida de la operación es idéntico pero la fuente de datos o las reglas de cálculo cambian.

- **Generics:**

  ¿Qué es Java Generics y cuáles son sus ventajas? ¿Cómo podría complementarse con el patrón plantilla para generalizar las clases Controlador y clases de Servicio?

  Los generics son una característica de Java que permiten parametrizar tipos (clases, interfaces y métodos), lo que significa que se puede escribir código que trabaje con diferentes tipos de datos (como objetos de tipo Cliente, Factura, Producto, etc.) manteniendo la seguridad de tipos en tiempo de compilación.

  Se definen utilizando un parámetro de tipo de esta forma: <T>. Cuando se utiliza la clase o el método, se indica qué tipo de dato va a manejar.

  Ventajas:

  - Seguridad de tipos: el compilador verifica los tipos en tiempo de compilación, previene errores como las excepciones ClassCastException en tiempo de ejecución.

  - Eliminación de castings: al usar colecciones o clases genéricas, ya no es necesario hacer conversiones manuales de tipos ((Cliente) objeto), ya que Java conoce el tipo exacto que se está manejando.

  - Reutilización de código: permite diseñar listas, repositorios o servicios base que sirven para cualquier entidad del negocio sin necesidad de duplicar código.

  La mayoría de las entidades de un sistema comparten las mismas operaciones del CRUD, y los controladores REST o web que manejan operaciones CRUD estándar repiten código para responder a las rutas HTTP. Por esto es posible crear clases que funcionan como plantillas, donde se define el orden de pasos a seguir para determinadas operaciones y lógica común pero dependiente del tipo de objeto, por lo que sin usar genéricos es necesario categorizar entidades y crear varias clases plantilla.
  El uso de genéricos en cambio permite implementar una única clase base de servicio y una de controlador para cualquier entidad.

- **Reflection:**

¿Qué es Java Reflection y cuáles son sus ventajas? Realizar en código de como se accede a los atributos privados de una clase y como se puede invocar un método.

Es una característica del lenguaje Java que permite inspeccionar, analizar y modificar el comportamiento interno de clases, interfaces, atributos, constructores y métodos durante el runtime, sin necesidad de conocer sus nombres o firmas en tiempo de compilación.

A través del paquete java.lang.reflect y la clase java.lang.Class, un programa puede examinar su propia estructura interna y romper barreras normales de encapsulamiento.

Ventajas:

- Permite instanciar clases y llamar métodos cargados dinámicamente mediante cadenas de texto
- Permite a frameworks como Spring Boot, Hibernate/JPA, Jackson y JUnit realizar tareas críticas como inyección de dependencias, mapeo objeto-relacional (ORM), serialización JSON y ejecución automática de pruebas anotadas.
- Permite leer metadatos (@Entity, @Autowired, @Override) en tiempo de ejecución para decidir la lógica a aplicar.

Java

```java
package com.ejemplo.reflection;

public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    // Método privado
    private void presentarse(String saludoInicial) {
        System.out.println(saludoInicial + ", soy " + this.nombre
+ " y tengo " + this.edad + " años.");
    }
}
```

Java

```java
package com.ejemplo.reflection;

import java.lang.reflect.Field;
import java.lang.reflect.Method;

public class MainReflection {

    public static void main(String[] args) {
        try {
            Persona persona = new Persona("Carlos", 28);
            Class<?> clazz = persona.getClass();

            // 1. acceder y modificar un atributo privado

            // getDeclaredField permite acceder a campos privados (a diferencia de getField)
            Field campoNombre = clazz.getDeclaredField("nombre");

            // Se desactiva la comprobación de acceso para omitir la restricción 'private'
            campoNombre.setAccessible(true);

            // Leer el valor privado actual
            String valorOriginal = (String) campoNombre.get(persona);
            System.out.println("Valor original del atributo 'nombre': " + valorOriginal);

            // Modificar el valor privado directamente en la instancia
            campoNombre.set(persona, "Matías");
            System.out.println("Nuevo valor del atributo 'nombre': " + campoNombre.get(persona));

            // 2. invocar un metodo privado con parametros

            // getDeclaredMethod recibe el nombre del método y los tipos de sus parámetros
            Method metodoPresentarse = clazz.getDeclaredMethod("presentarse", String.class);

            // Habilitar acceso al método privado
            metodoPresentarse.setAccessible(true);
```

```
            // Invocación: primer argumento es la instancia objetivo,
seguido de los parámetros
            metodoPresentarse.invoke(persona, "¡Hola a todos");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```